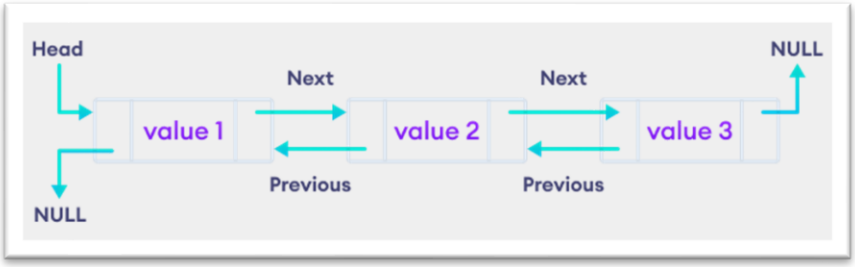
# Lista dwukierunkowa

Klasa szablonu opisuje obiekt, który kontroluje różną długość sekwencji elementów, które mają dostęp dwukierunkowy. Kontener służy list do zarządzania sekwencją elementów jako dwukierunkową połączoną listą węzłów, z których każdy przechowuje jeden element.

Lista "List" to kontener sekwencyjny. Kontener ten charakteryzuje się stałym czasem wstawiania lub usuwania elementu z obu kierunków. Kontener ten zaimplementowany jest jako lista dwukierunkowa. Każdy element listy zawiera wskaźnik na następny i poprzedni obiekt elementu listy, dzięki czemu kolejne elementy nie są powiązane ze sobą miejscem alokacji.




## Budowa list dwukierunkowych

W Standardowej Bibliotece Szablonów (STL) w C++ lista dwukierunkowa jest reprezentowana przez klasę szablonową std::list. Jest to kontener sekwencyjny, który implementuje listę dwukierunkową, co oznacza, że każdy element listy przechowuje wskaźniki do następnego i poprzedniego elementu.

## Cechy budowy:

- Dynamiczna alokacja pamięci – elementy listy nie są przechowywane w ciągłej lokalizacji pamięci.
- Wskaźniki na sąsiednie elementy – każdy element posiada wskaźnik na następny i poprzedni element.
- O(n) dostęp do elementów – przeglądanie listy wymaga iteracji od początku lub końca (brak dostępu losowego, jak w std::vector).
- Szybkie wstawianie i usuwanie – operacje dodawania lub usuwania elementów w dowolnym miejscu mają złożoność $O(1)$, jeśli mamy iterator na dany element.

## Zastosowanie list dwukierunkowych

- Wstawianie i usuwanie elementów: std::list jest idealna do scenariuszy, gdzie często trzeba dodawać lub usuwać elementy z dowolnego miejsca, w tym z początku i końca.
- Funkcje jak push_front(), push_back(), pop_front(), pop_back() zapewniają wydajną obsługę tych operacji.
- Kolejki i stosy: Dzięki wsparciu operacji na początku i końcu, lista dwukierunkowa może być wykorzystana do implementacji struktur takich jak stosy (LIFO) i kolejki (FIFO).
- Algorytmy i manipulacje danych: std::list obsługuje algorytmy STL (np. sortowanie sort(), usuwanie remove(), unikalne wartości unique()), które mogą być wykonywane bez konieczności kopiowania danych.
- Iteracja w obu kierunkach Dwukierunkowe iteratory (std::list::iterator) umożliwiają poruszanie się zarówno w przód, jak i wstecz, co jest przydatne w niektórych algorytmach.
- Realizacja struktur danych: Lista dwukierunkowa jest podstawą bardziej zaawansowanych struktur, takich jak deki, kolejki priorytetowe czy algorytmy grafowe.

## Właściwości kontenera

- Implementacja kontenera list jest podobna do kontenera forward list. Różnica między nimi polega na tym, że kontener forward list posiada wskaźnik tylko do następnego elementu kontenera przez co iteracja może odbywać się jedynie w przód;
- Operacje wstawiania, wydobywania i przenoszenia są szybsze niż w przypadku kontenerów: array, vector i deque. Zaleta ta sugeruje, że w przypadku algorytmów sortujących zastosowanie kontenera listy może zwiększyć efektywność programu;
- Bezpośredni dostęp do elementów listy jest niemożliwy. W celu dostania się do konkretnego elementu należy przeiterować kontener określoną ilość razy.

```
1  #include<iostream>
2  // include do listy
3  #include<list>
4  using namespace std;
5
6  int main()
7  {setlocale(LC_CTYPE, "Polish");
8
9  // tworzenie listy
10 list<int>lista {1,2,3,4,5};
11
12 cout<<"Elementy listy:"<<endl;
13 // wyświetlenie listy
14 for(int numer:lista){cout<<numer<<" ";}
15
16
17 return 0;}
```

## Wady i zalety listy

Zalety

- szybkie dodawanie i usuwanie elementów,
- możliwość przejścia w obu kierunkach,
- działa dynamicznie – rozmiar może się zmieniać,
- dobre do systemów z kolejkami zadań, symulacji, algorytmów grafowych.

Wady

- brak dostępu przez indeksy (L[3] – niedozwolone) – tylko iteratory
- wolniejsze przeszukiwanie niż w vector,
- większe zużycie pamięci.

## Typowe zastosowania

- systemy kolejkowania zadań,
- implementacja historii cofania (undo/redo),
- algorytmy przeszukiwania grafów,
- zarządzanie dynamicznymi listami elementów.

## Operacje na liście

### empty

Zwraca true jeżeli lista jest pusta lub false jeżeli nie jest.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista;
9
10 if(lista.empty()==true)
11  {cout<<"lista jest pusta"<<endl;}
12 else
13  {cout<<"lista NIE jest pusta"<<endl;}
14
15 cout<<"wstawiam do listy 10"<<endl;
16 lista.push_back(10);
17
18 if(lista.empty()==true)
19  {cout<<"lista jest pusta"<<endl;}
20 else
21  {cout<<"lista NIE jest pusta"<<endl;}
22
23 return 0;}
```

### size

Zwraca ilość elementów w liście.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista;
9
10 cout<<"wstawiam do listy 10"<<endl;
11 lista.push_back(10);
12 cout<<"wstawiam do listy 20"<<endl;
13 lista.push_back(20);
14
15 cout<<"Ilość elementów w liście:"<<lista.size();
16
17 return 0;}
```

### resize

Zmienia rozmiar listy.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,2,3,4,5,6};
9
10 for(int numer:lista){cout<<numer<<" ";}
11 cout<<endl<<"Ilość elementów w liście:"<<lista.size()<<endl;
12 //zmiana rozmiaru listy na 3
13 lista.resize(3);
14 for(int numer:lista){cout<<numer<<" ";}
15 cout<<endl<<"Ilość elementów w liście:"<<lista.size();
16
17 return 0;}
```

```
1 2 3 4 5 6
Ilość elementów w liście:6
1 2 3
Ilość elementów w liście:3
--------------------------------
```

```
#include<iostream>
#include<list>
using namespace std;

int main()
{setlocale(LC_CTYPE, "Polish");

list<int>lista={1,2,3,4,5,6};

for(int numer:lista){cout<<" "<<numer;}
cout<<endl<<" Ilość elementów w liście:"<<lista.size()<<endl;
//zmiana rozmiaru listy na 10, uzupełnienie wart. 20
lista.resize(10,20);
for(int numer:lista){cout<<" "<<numer;}
cout<<endl<<" Ilość elementów w liście:"<<lista.size();

return 0;}
```

```
 1 2 3 4 5 6
 Ilość elementów w liście:6
 1 2 3 4 5 6 20 20 20 20
 Ilość elementów w liście:10
--------------------------------
Process exited after 0.1002 seconds with return value 0
Press any key to continue . . .
```

## max_size

Zwraca maksymalną ilość elementów możliwych do zapisania w liście.

```
#include<iostream>
#include<list>
using namespace std;

int main()
{setlocale(LC_CTYPE, "Polish");

list<int>lista;
unsigned int i;

cout<<" podaj ile elementów ma mieć lista :"<<endl;
cin>>i;
if(i<lista.max_size()) {
 lista.resize(i,123);}
else {
 cout<<"Przekraczasz maksymalny rozmiar listy"<<endl;
 cout<<"Maksymalny rozmiar listy to: "<<lista.max_size()<<endl;}

 cout<<endl<<" Ilość elementów w liście:"<<lista.size()<<endl;

return 0;}
```

```
podaj ile elementów ma mieć lista :
99999999

 Ilość elementów w liście:99999999

--------------------------------
Process exited after 22.25 seconds with return value 0
Press any key to continue . . .
```

## front

Dostęp do pierwszego elementu listy.

```
#include<iostream>
#include<list>
using namespace std;

int main()
{setlocale(LC_CTYPE, "Polish");

list<int>lista={1,2,3,4,5,6};

for(int numer:lista){cout<<" "<<numer;}
// 1 2 3 4 5 6
cout<<endl<<lista.front()<<endl; // 1
lista.front()=5;
for(int numer:lista){cout<<" "<<numer;}
//5 2 3 4 5 6

return 0;}
```

## back

Dostęp do ostatniego elementu listy.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,2,3,4,5,6};
9
10 for(int numer:lista){cout<<" "<<numer;}
11 // 1 2 3 4 5 6
12 cout<<endl<<lista.back()<<endl; // 6
13 lista.back()=1;
14 for(int numer:lista){cout<<" "<<numer;}
15 //1 2 3 4 5 1
16
17 return 0;}
```

## assign

Przypisuje nową zawartość do kontenera listy, zastępując jego obecną zawartość i modyfikując odpowiednio jego rozmiar.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista1;
9  list<int>lista2;
10
11 lista1.assign(4,10);
12
13 for(int numer:lista1){cout<<" "<<numer;} cout<<endl;
14 // 10 10 10 10
15
16 lista2.assign(lista1.begin(), lista1.end());
17 for(int numer:lista2){cout<<" "<<numer;} cout<<endl;
18 // 10 10 10 10
19
20 int tab[5]={1,1,2,2,3};
21 lista1.assign(tab,tab+3);
22 for(int numer:lista1){cout<<" "<<numer;} cout<<endl;
23 // 1 1 2
24 return 0;}
```

## push_front

Wstawia nowy element na początku listy, tuż przed jej obecnym pierwszym elementem.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,1,2,2};
9  lista.push_front(10);
10 lista.push_front(12);
11
12 for(int numer:lista){cout<<" "<<numer;} cout<<endl;
13 //12 10 1 1 2 2
14 return 0;}
```

## push_back

Wstawia nowy element na końcu listy, tuż za jej obecnym ostatnim elementem.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,1,2,2};
9  lista.push_back(10);
10 lista.push_back(12);
11
12 for(int numer:lista){cout<<" "<<numer;} cout<<endl;
13 // 1 1 2 2 10 12
14 return 0;}
```

## pop_front

Usuwa pierwszy element z kontenera listy, zmniejszając jego rozmiar o jeden.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,2,3,4};
9  lista.pop_front();
10 for(int numer:lista){cout<<" "<<numer;} cout<<endl;
11 // 2 3 4
12 return 0;}
```

## pop_back

Usuwa ostatni element z kontenera listy, zmniejszając jego rozmiar o jeden.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,2,3,4};
9  lista.pop_back();
10 for(int numer:lista){cout<<" "<<numer;} cout<<endl;
11 // 1 2 3
12 return 0;}
```

## emplace

Kontener jest rozszerzany poprzez wstawienie nowego elementu na pozycję (iterator, begin lub end). Zwiększa rozmiar listy o jeden.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,2,3,4};
9  list<int>::iterator it;
10
11 it=lista.begin();
12 it++;
13
14 int a=100;
15 lista.emplace(it,a);
16 lista.emplace(lista.end(),a);
17 for(int numer:lista){cout<<" "<<numer;} cout<<endl;
18 // 1 100 2 3 4 100
19 return 0;}
```

## insert

Kontener jest rozszerzany poprzez wstawienie nowego elementu na pozycję (iterator, begin lub end). Zwiększa rozmiar listy.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,2,3,4};
9  list<int>::iterator it;
10
11 it=lista.begin();
12 it++; // pokazuje na 2 pozycję
13
14 int a=100;
15
16 lista.insert(it,a);
17 for(int numer:lista){cout<<" "<<numer;} cout<<endl;
18 // 1 100 2 3 4
19 it=lista.end();
20 --it; // pokazuje na przed ostatnią pozycje
21
22 lista.insert(it,2,20);
23 for(int numer:lista){cout<<" "<<numer;} cout<<endl;
24 // 1 100 2 3 20 20 4
25
26 return 0;}
```

## erase

Usuwa z kontenera listy pojedynczy element (pozycję) lub zakres elementów ([pierwszy, ostatni)).

```
#include<iostream>
#include<list>
using namespace std;

int main()
{setlocale(LC_CTYPE, "Polish");

list<int>lista={10,20,30,40,50,60,70,80,90};
list<int>::iterator it1, it2;



                                  // 10 20 30 40 50 60 70 80 90
  it1 = it2 = lista.begin();      // ^^
  advance (it2,6);                // ^                 ^
  ++it1;                          //    ^              ^

  it1 = lista.erase (it1);        // 10 30 40 50 60 70 80 90
                                  //    ^           ^

  it2 = lista.erase (it2);        // 10 30 40 50 60 80 90
                                  //    ^           ^

  ++it1;                          //       ^        ^
  --it2;                          //       ^     ^

  lista.erase (it1,it2);          // 10 30 60 80 90
                                  //        ^


for(int numer:lista){cout<<" "<<numer;} cout<<endl;
// 10 30 60 80 90

return 0;}
```

## swap

Zamienia zawartość kontenera na zawartość x, który jest inną listą tego samego typu. Rozmiary mogą się różnić.

```
#include<iostream>
#include<list>
using namespace std;

int main()
{setlocale(LC_CTYPE, "Polish");

list<int>lista1={1,2,3,4,5,6};  // 6 elementów
list<int>lista2={10,20,30};     // 3 elementy

cout<<endl<<" Ilość elementów w lista1:"<<lista1.size()<<endl;
// 6
cout<<endl<<" Ilość elementów w lista2:"<<lista2.size()<<endl;
// 3
lista1.swap(lista2);

cout<<endl<<" Ilość elementów w lista1:"<<lista1.size()<<endl;
// 3
cout<<endl<<" Ilość elementów w lista2:"<<lista2.size()<<endl;
// 6
return 0;}
```

## resize

Zmienia rozmiar kontenera tak, aby zawierał n elementów. Jeśli n jest mniejsze niż aktualny rozmiar kontenera, zawartość zostaje zmniejszona do pierwszych n elementów, usuwając te poza nimi (i niszcząc je). Jeśli n jest większe niż aktualny rozmiar kontenera, zawartość jest rozszerzana poprzez wstawienie na końcu tylu elementów, ile jest potrzebne, aby osiągnąć rozmiar n. Jeśli val jest określone, nowe elementy są inicjalizowane jako kopie val, w przeciwnym razie są inicjalizowane wartościami domyślnymi.

```
1   #include<iostream>
2   #include<list>
3   using namespace std;
4
5   int main()
6   {setlocale(LC_CTYPE, "Polish");
7
8   list<int>lista={1,2,3,4,5,6};
9
10  lista.resize(3);
11  // 1 2 3
12  lista.resize(5,100);
13  // 1 2 3 100 100
14  lista.resize(7);
15  // 1 2 3 100 100 0 0
16
17  return 0;}
```

## clear

Usuwa wszystkie elementy z kontenera listy (które są niszczone) i pozostawia kontener o rozmiarze 0.

```
1   #include<iostream>
2   #include<list>
3   using namespace std;
4
5   int main()
6   {setlocale(LC_CTYPE, "Polish");
7
8   list<int>lista={1,2,3,4,5,6};
9   // 1 2 3 4 5 6
10  lista.clear();
11  //
12  lista.push_back(10);
13  // 10
14
15  return 0;}
```

## remove

Usuwa z kontenera wszystkie elementy, które są równe **val**. Wywołuje destruktor tych obiektów i zmniejsza rozmiar kontenera o liczbę usuniętych elementów. W przeciwieństwie do funkcji składowej list::erase, która usuwa elementy według ich pozycji (za pomocą iteratora), ta funkcja (list::remove) usuwa elementy według ich wartości. Istnieje podobna funkcja, list::remove_if, która pozwala na określenie warunku innego niż porównanie równości w celu określenia, czy element ma zostać usunięty.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  int main()
6  {setlocale(LC_CTYPE, "Polish");
7
8  list<int>lista={1,2,3,1,2,3,1};
9  lista.remove(1);
10 // 2 3 2 3
11
12 return 0;}
```

## remove_if

Usuwa z kontenera wszystkie elementy, które są równe **val**. Wywołuje destruktor tych obiektów i zmniejsza rozmiar kontenera.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  bool parzyste (const int& x) { return (x%2==0); }
6
7  int main()
8  {setlocale(LC_CTYPE, "Polish");
9
10 list<int>lista={1,2,3,1,2,3,1};
11 lista.remove_if(parzyste);
12 // 1 3 1 3 1
13
14 return 0;}
```

## reverse

Odwraca kolejność elementów w kontenerze listy.

```
1  #include<iostream>
2  #include<list>
3  using namespace std;
4
5  bool parzyste (const int& x) { return (x%2==0); }
6
7  int main()
8  {setlocale(LC_CTYPE, "Polish");
9
10 list<int>lista={1,2,3,4,5,6};
11 lista.reverse();
12
13 // 6 5 4 3 2 1
14
15 return 0;}
```

## sort

Sortuje elementy listy w kolejności rosnącej. Kolejność równych elementów jest zachowana.

```
1   #include<iostream>
2   #include<list>
3   using namespace std;
4
5   int main()
6   {setlocale(LC_CTYPE, "Polish");
7
8   list<int>lista={1,6,2,5,3,4};
9   lista.sort();
10  // 1 2 3 4 5 6
11
12  return 0;}
```

## merge

merge() jest wbudowaną funkcją w C++ STL, która jest zadeklarowana w pliku nagłówkowym <list>. merge() służy do scalania dwóch list w jedną. Możemy po prostu scalić dwie listy, lub jeżeli isty są posortowane – połączy je sortując obie.

```
1   #include<iostream>
2   #include<list>
3   using namespace std;
4
5   int main()
6   {setlocale(LC_CTYPE, "Polish");
7   // listy nieposortowane
8   list<int>lista1={1,6,2,5,3,4};
9   list<int>lista2={7,9,8,10};
10  lista1.merge(lista2);
11  // 1 6 2 5 3 4 7 9 8 10
12
13  return 0;}
```

```
1   #include<iostream>
2   #include<list>
3   using namespace std;
4
5   int main()
6   {setlocale(LC_CTYPE, "Polish");
7   // listy posortowane
8   list<int>lista1={1,4,6,8};
9   list<int>lista2={2,3,5,7};
10  lista1.merge(lista2);
11  //  1 2 3 4 5 6 7 8
12
13  return 0;}
```

## unique

Funkcja unique() w strukturze list służy do usuwania kolejnych powtarzających się elementów, które znajdują się obok siebie. Usuwa tylko duplikaty sąsiadujące, a nie wszystkie wystąpienia danej wartości w całej liście.

```
1  #include <iostream>
2  #include <list>
3  using namespace std;
4
5  int main() {
6      setlocale(LC_CTYPE, "Polish");
7      list<int> L = {1, 2, 2, 3, 2, 2, 4};
8      L.unique();
9      for(int x : L) {cout<<x<<" ";}
10     // 1 2 3 2 4
11     return 0;
12 }
```

## Ćwiczenie 1

Napisz program, który:

1. tworzy listę typu int,
2. dodaje kilka liczb (na początek i na koniec),
3. wyświetla wszystkie elementy listy,
4. usuwa jeden wskazany element (np. wartość 20),
5. ponownie wyświetla listę.

## Ćwiczenie 2

Utwórz listę string.
Program powinien:

- pobierać od użytkownika zdania do listy (do wpisania „stop”),
- wyświetlać liczbę elementów,
- usunąć ostatni dodany element,
- wyświetlić zawartość listy po usunięciu.

## Ćwiczenie 3

Pobierz od użytkownika 10 liczb typu int i umieść je na liście.
Następnie:

- posortuj listę metodą sort(),
- zapytaj użytkownika o liczbę do wyszukania,
- sprawdź, czy ta liczba znajduje się w liście,
- wyświetl komunikat „znaleziono” lub „nie znaleziono”.